Graph Algorithms

- Chapter 23 of CLR
    - Minimum Spanning Tree
    - Prim
    - Kruskal

    - ***Chapter 21***

Minimum Spanning Tree

Input: a connected, undirected graph G = (V, E) with weights w: E → R on the edges

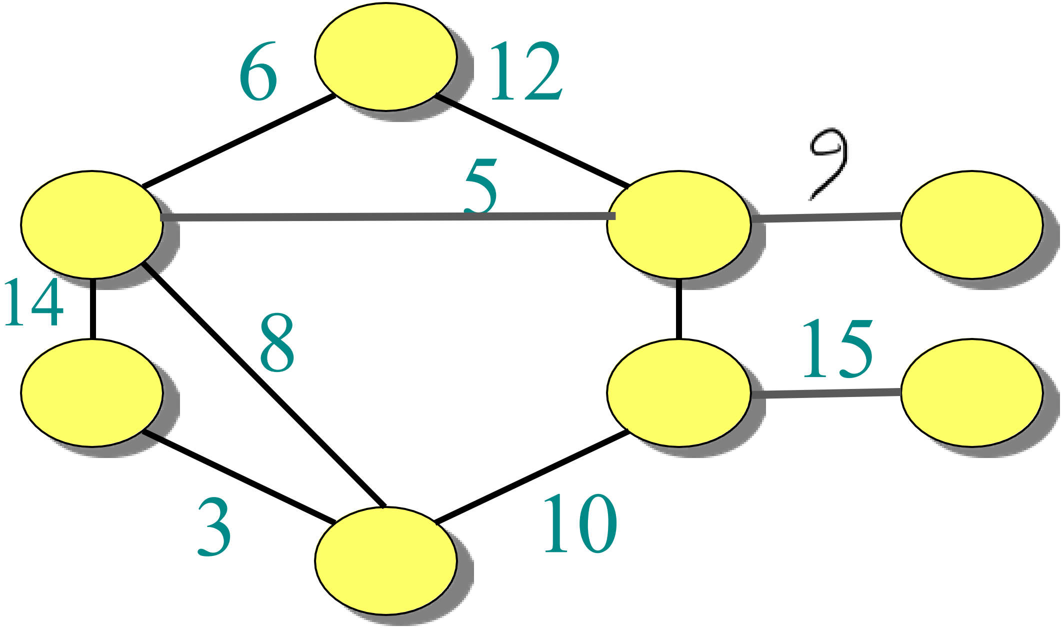
Output: a minimum spanning tree T

- A **spanning tree** of G is a graph (V, T ⊆ E) such that (V,T) is a tree
    - A tree: a connected graph with no cycle
- The weight of a tree:

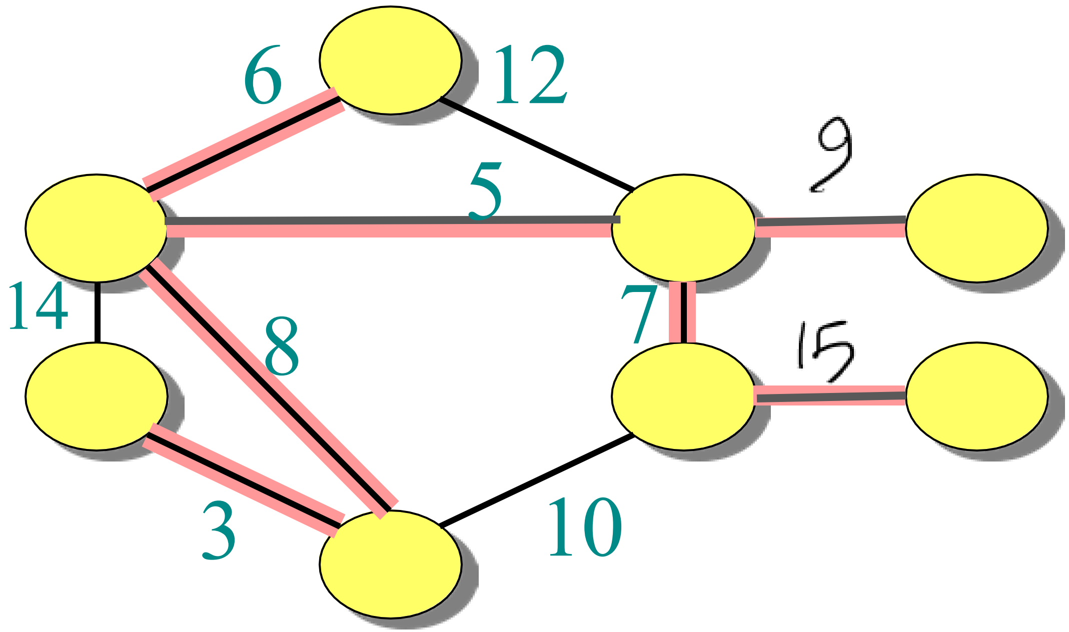
$$w(T) = \sum_{(u,v)\in T} w(u,v)$$

- A **minimum spanning tree**: a tree of minimum weight:
        - subset of edges (of size n − 1) that connects all the vertices and has minimum weight

Example of MST


6
12
5
9
14
8
15
3
10

Example of MST


6
12
5
9
14
8
7
15
3
10

The edges on
spanning tree ▬

The weight of the above tree is 6+5+8+3+7+9+15

Minimum Spanning Trees

There are many greedy algorithms for finding MSTs. We will cover the followings:

- Kruskal's algorithm (1956)
- Prim's algorithm (1930, rediscovered 1957)

We will explore Kruskal's algorithm and Prim's algorithm in this course.

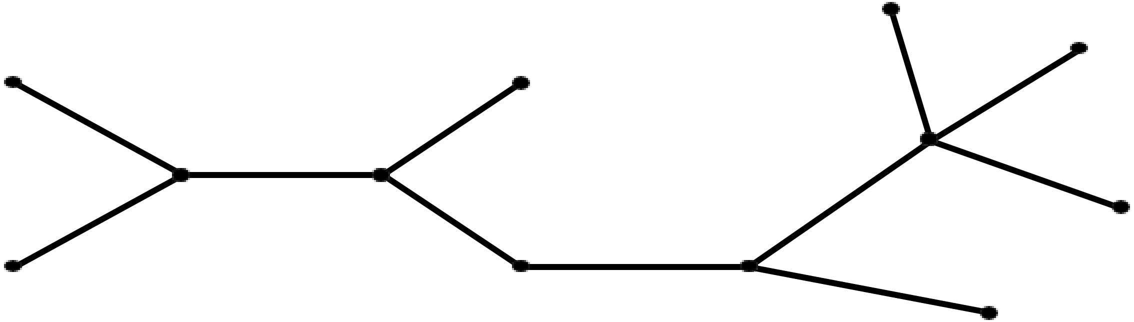
Minimum Spanning Tree

- Can there be more than one minimum spanning tree (MST) for an undirected graph?
    - Yes
- What happens if the graph is unweighted?
    - All spannings trees are minimum spanning trees

Optimal substructure

MST *T*:

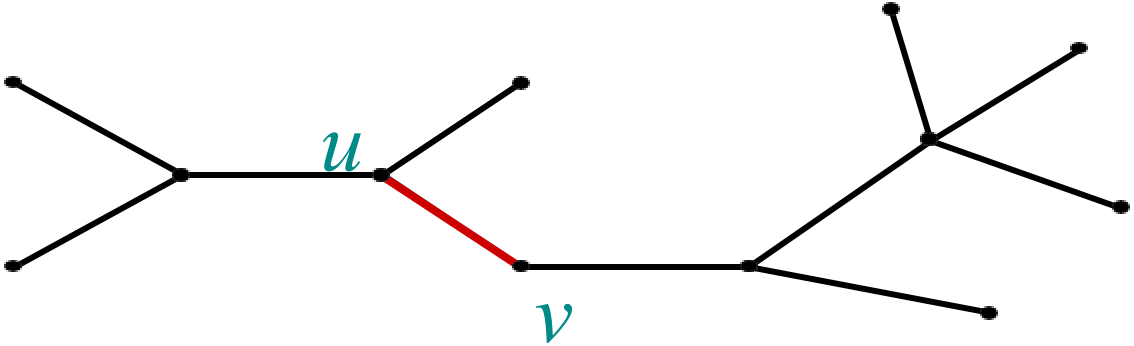
(Other edges of *G* are not shown.)



Optimal substructure

MST T:

(Other edges of G are not shown.)

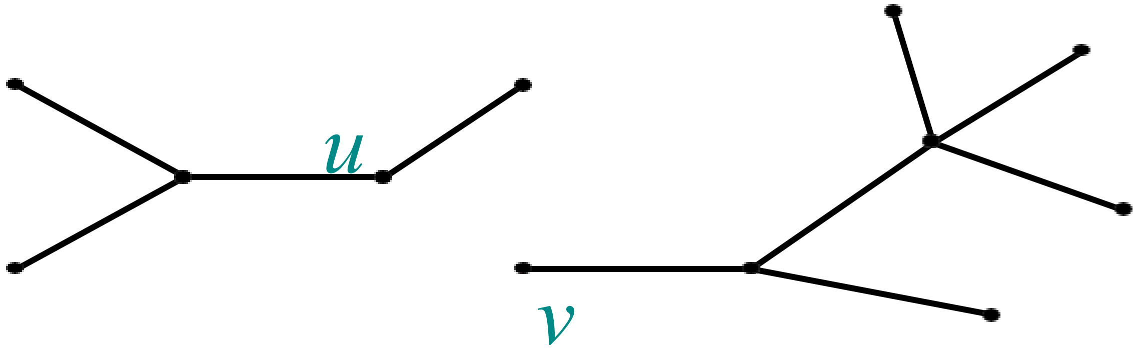


Remove any edge $(u, v) \in T$.

Optimal substructure

MST T:

(Other edges of G are not shown.)

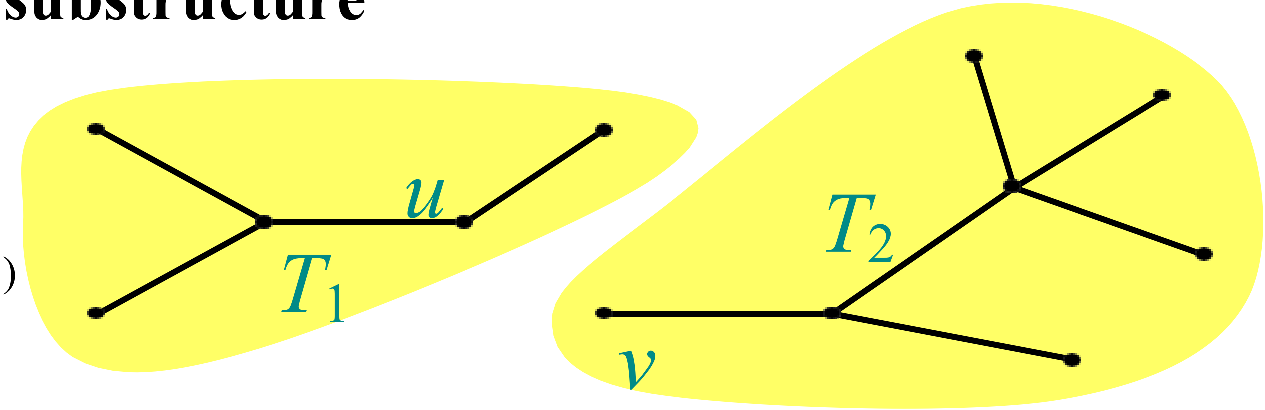


Remove any edge $(u, v) \in T$.

Optimal substructure

MST T:

(Other edges of G are not shown.)



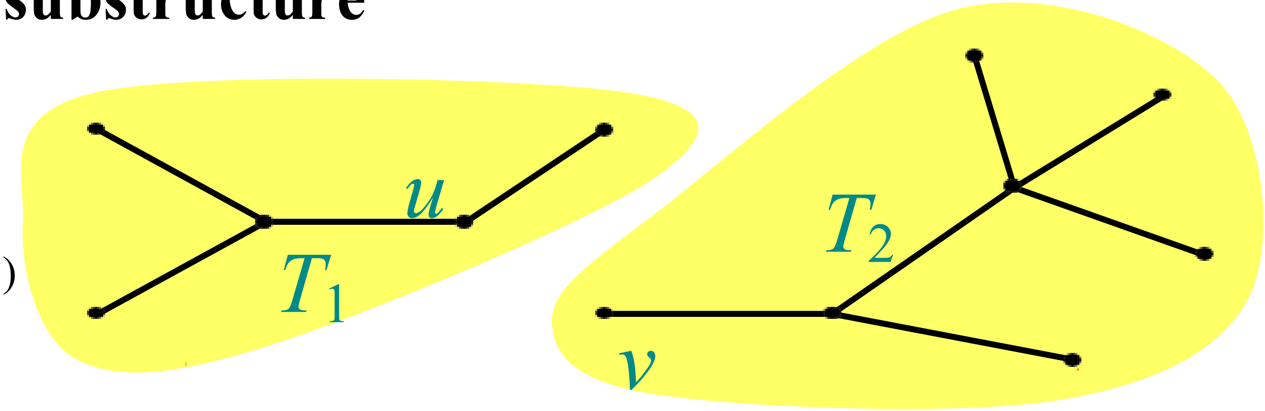
Remove any edge $(u, v) \in T$.

Then, T is partitioned into two subtrees T_1 and T_2.

Optimal substructure

MST T:

(Other edges of G are not shown.)



Remove any edge $(u, v) \in T$.

Then, T is partitioned into two subtrees T_1 and T_2.

Theorem. The subtree T_1 is MST of $G_1 = (V_1, E_1)$, the subgraph of G ***induced*** by the vertices of T_1:

$$V_1 = \text{vertices of } T_1,$$
$$E_1 = \{ (x, y) \in E : x, y \in V_1 \}.$$

Similarly for T_2.

Proof of optimal substructure property for MST problem

Proof. Cut and paste:

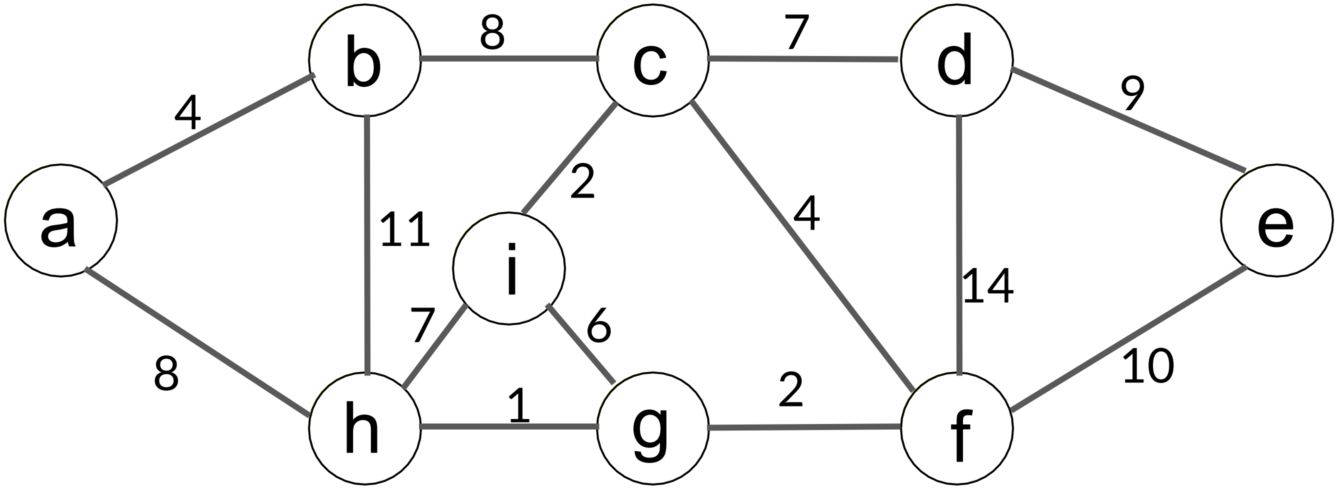
$$w(T) = w(u, v) + w(T_1) + w(T_2).$$

If T_1' were a lower-weight spanning tree than T_1 for G_1, then $T' = \{(u, v)\} \cup T_1' \cup T_2$ would be a lower-weight spanning tree than T for G.

Contradiction: since T was the minimum spanning tree for G

Kruskal Algorithm

```
T= Ø
Repeat
    ● find the least-weight edge (u,v) so that u and v are not connected in T
    ● add (u,v) to T
```

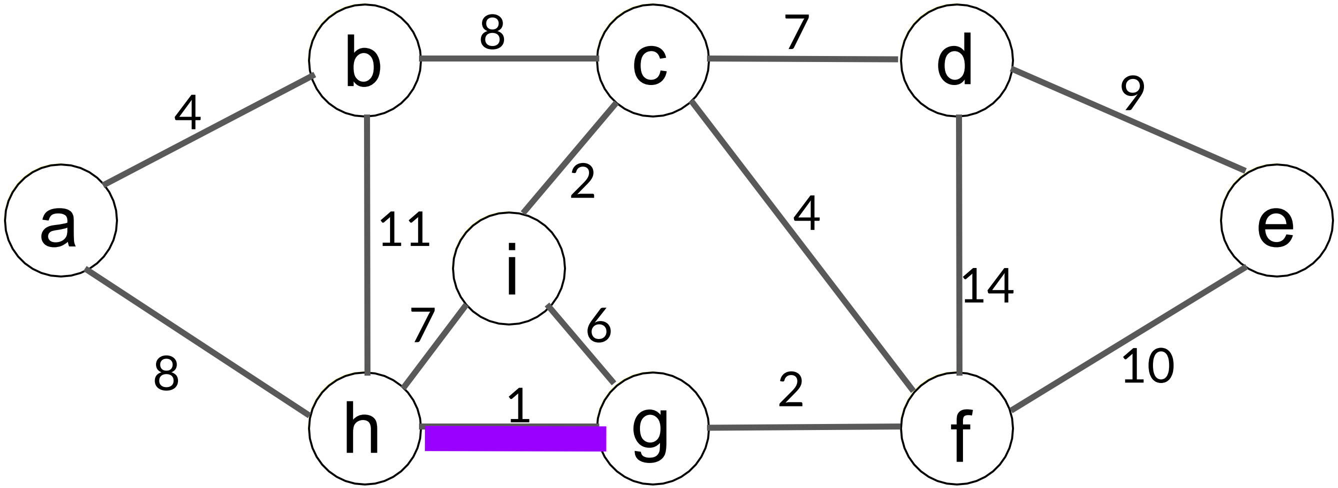


a
b
c
d
e
f
g
h
i
4
8
8
7
9
11
2
4
14
7
6
10
1
2

Kruskal Algorithm

T= Ø
Repeat

- find the least-weight edge (u,v) so that u and v are not connected in T
- add (u,v) to T

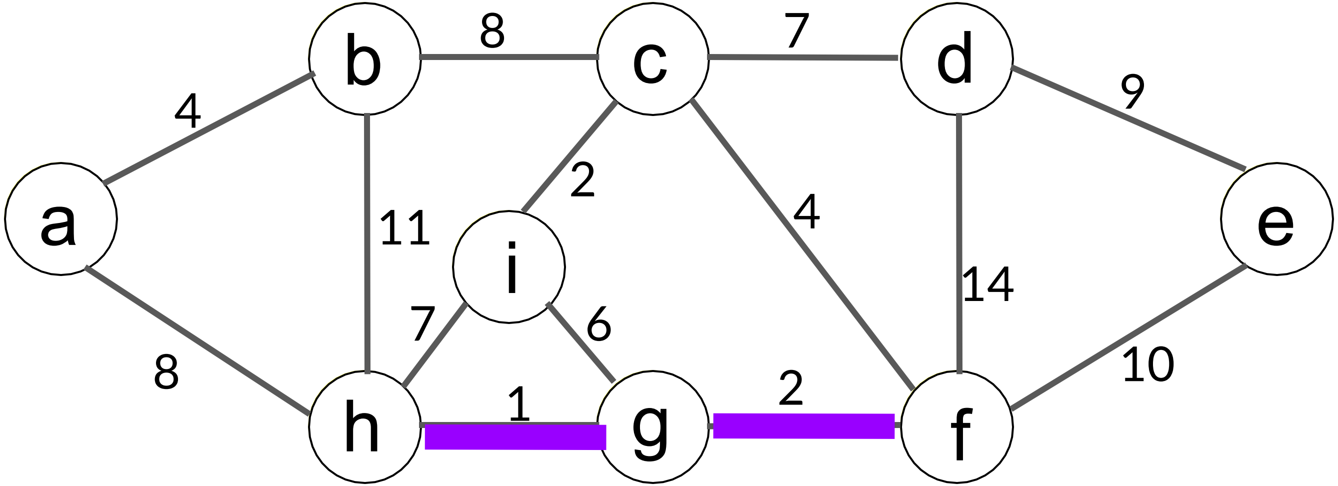

b
8
c
7
d
4
9
a
11
2
4
e
i
14
7
6
8
2
10
1
h
g
f

Kruskal Algorithm

T= Ø

Repeat

- find the least-weight edge (u,v) so that u and v are not connected in T
- add (u,v) to T



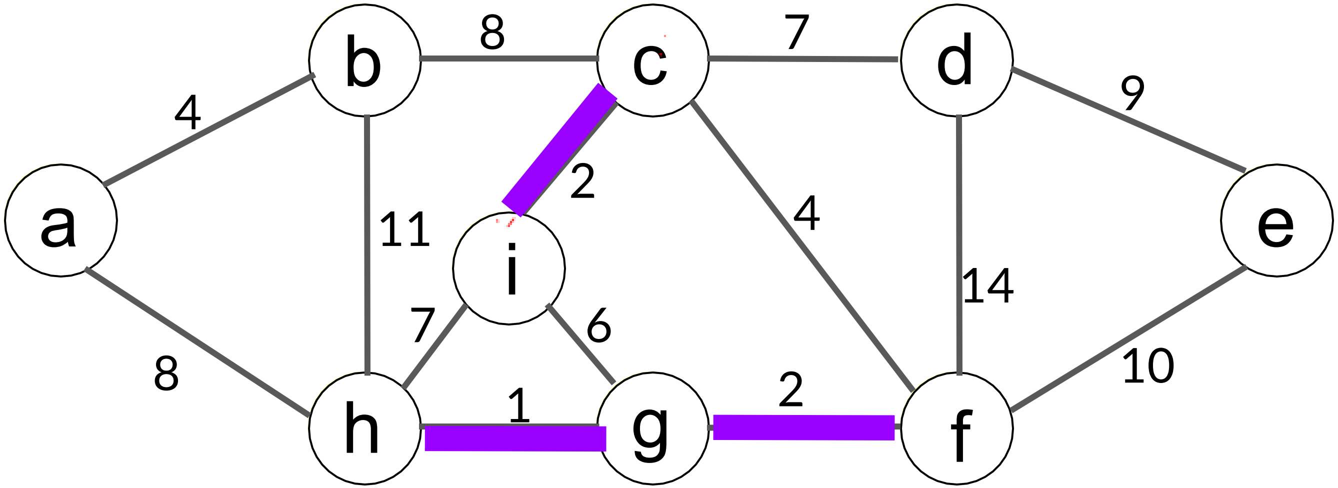
a
b
c
d
e
f
g
h
i
4
8
8
11
7
2
4
9
14
7
6
1
2
10

Kruskal Algorithm

T= Ø

Repeat

- find the least-weight edge (u,v) so that u and v are not connected in T
- add (u,v) to T



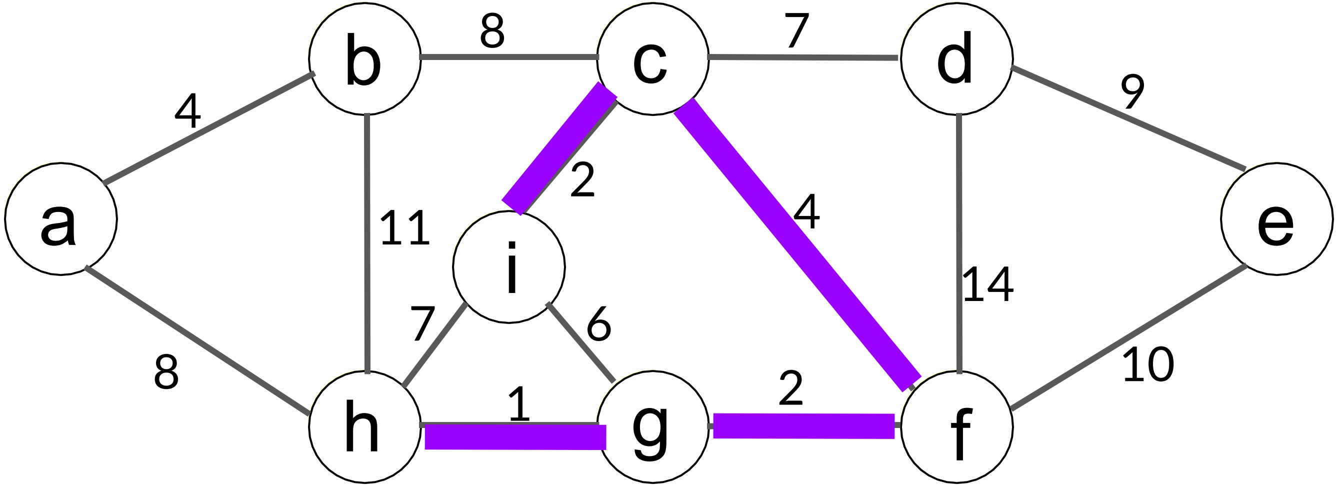
a
b
c
d
e
f
g
h
i
4
8
8
11
2
7
4
9
14
7
6
1
2
10

Kruskal Algorithm

T= Ø

Repeat

- find the least-weight edge (u,v) so that u and v are not connected in T
- add (u,v) to T

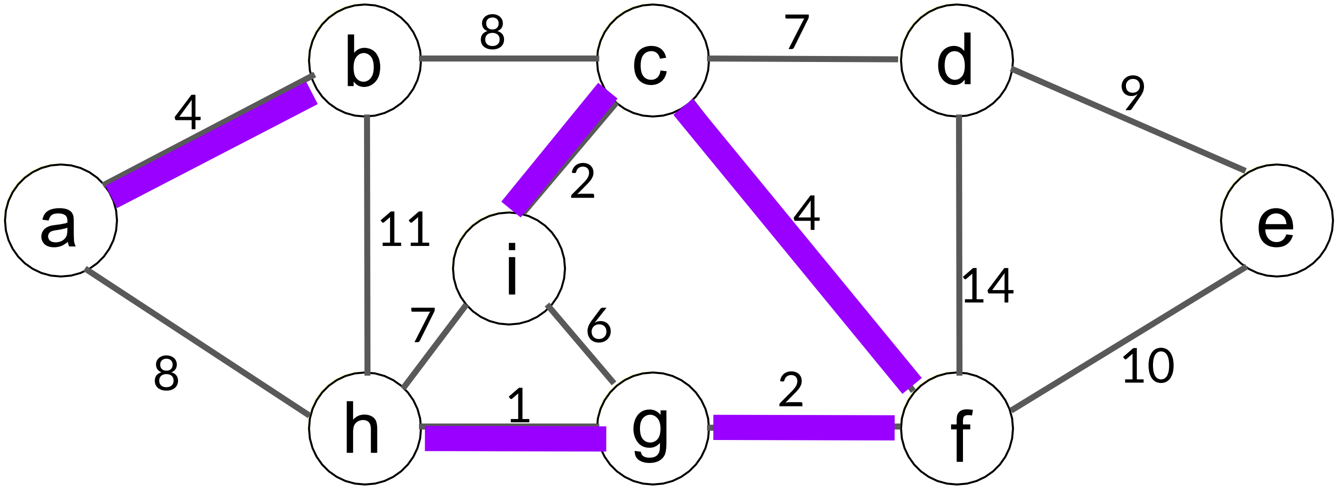

a
b
c
d
e
f
g
h
i
4
8
8
11
7
1
2
2
4
6
7
9
14
10

Kruskal Algorithm

T= Ø

Repeat

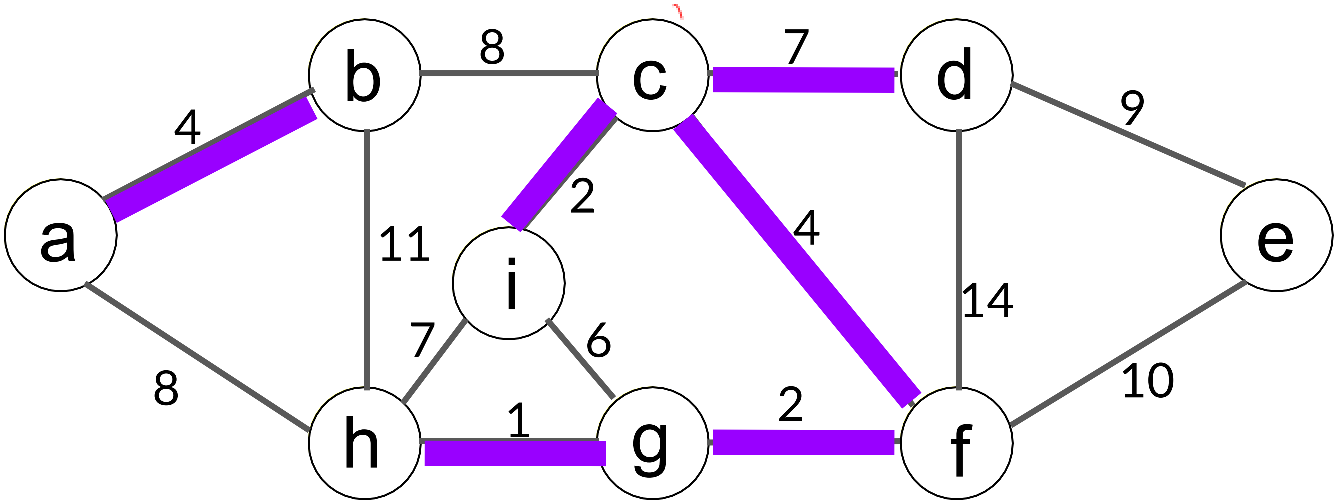
- find the least-weight edge (u,v) so that u and v are not connected in T
- add (u,v) to T



b
8
c
7
d
4
9
2
a
11
i
4
e
14
7
6
8
2
10
h
1
g
f

Kruskal Algorithm

```
T= Ø
Repeat
  ● find the least-weight edge (u,v) so that u and v are not connected in T
  ● add (u,v) to T
```



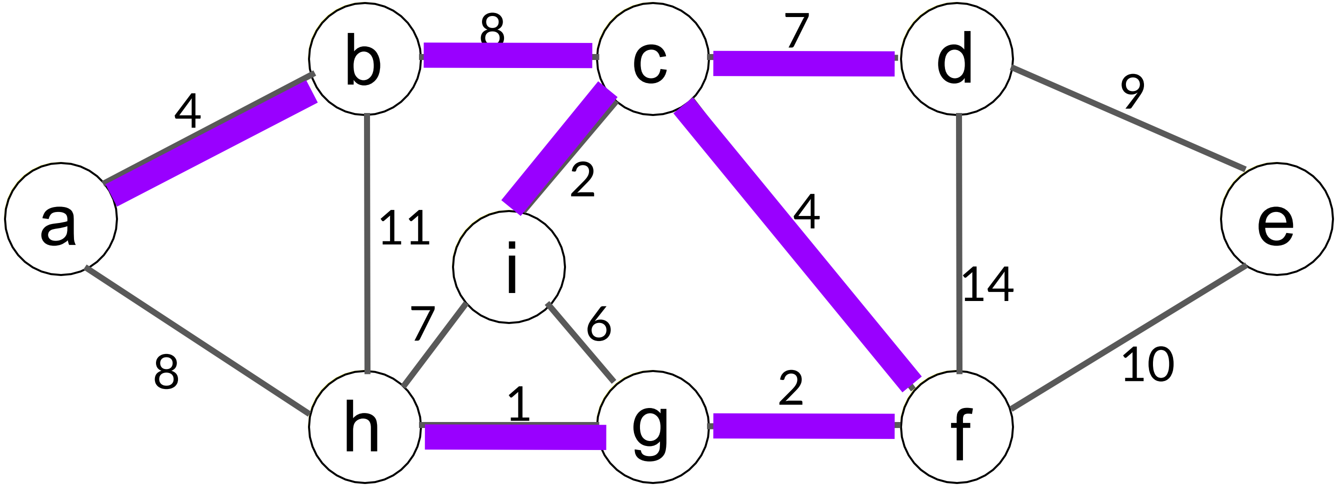
a
b
c
d
e
f
g
h
i
4
8
7
9
11
2
4
14
7
6
8
1
2
10

Kruskal Algorithm

T= Ø

Repeat

- find the least-weight edge (u,v) so that u and v are not connected in T
- add (u,v) to T

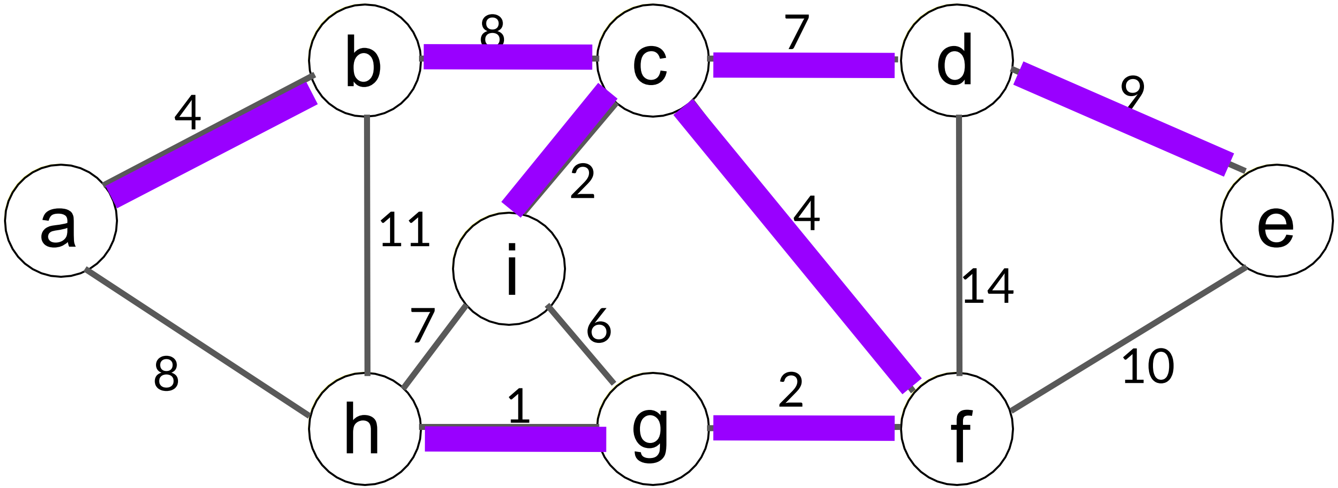

a
b
c
d
e
f
g
h
i
4
8
8
11
7
2
4
7
9
14
10
6
7
1
2

Kruskal Algorithm

T= Ø

Repeat

- find the least-weight edge (u,v) so that u and v are not connected in T
- add (u,v) to T

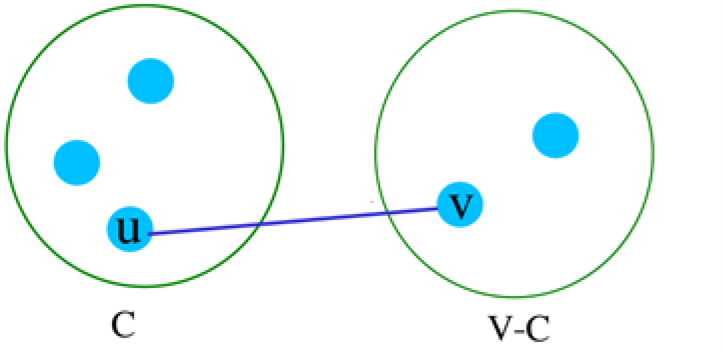


a
b
c
d
e
f
g
h
i
4
8
7
9
11
2
4
14
10
7
6
8
1
2

Graph Cuts

In a graph G = (V, E)

- A **cut** is a partition of the vertices of the graph into two sets C, V-C
    - We show it by (C, V-C)
    - C ⊆ V.
- An edge (u,v) crosses the cut (C, V-C) if exactly one of u, v is in C
- If G is connected, then at least one edge crosses every cut



Tree Facts

- A tree of n vertices has n-1 edges
- There is a unique path between any two vertices in a tree
- If T is a tree and an edge e ∉ T is added to T, then the resulting graph contains a unique cycle C
- If e’ ∈ C then T U {e} \ {e’} is a tree
    - If you add an edge e to a tree and this creates a cycle C, then removing any other edge e’ ∈C will break the cycle and produce a tree
    - Proof in the next slide

Tree Facts

Theorem. Let T be a tree and $e=(u, v) \notin T$. The graph $T \cup \{e\}$ contains a cycle. For any edge $e'=(x, y)$ on the cycle, the graph $T' = T \cup \{e\} - \{e'\}$ is a tree.

Proof.

Tree Facts

Theorem. Let T be a tree and e=(u, v) ∉ T. The graph T ∪ {e} contains a cycle. For any edge e'=(x, y) on the cycle, the graph T' = T ∪ {e} – {e'} is a tree.

Proof.

- |E'| = |E| + 1 – 1 = |E| = |V| – 1 →if T' is connected, then it is a tree. Why?
  - e ∉ T and e' ∈ T ∪ {e}
- Proving T' is connected
  - Consider any s, t ∈ V. Since T is connected, there is some path from s to t in T.
    - If that path does not cross (x, y), or if (x, y) = (u, v), then this path is also a path from s to t in T', so s and t are connected in T'.
    - If the path from s to t crosses (x, y). Assume WLOG that the path starts at s, goes to x, crosses (x, y), then goes from y to t. Since (u, v) and (x, y) are part of the same cycle, we can modify the original path from s to t so that instead of crossing (x, y), it goes around the cycle from x to y. This new path is then a path from s to t in T', so s and t are connected in T'. Thus any arbitrary pair of nodes are connected in T', so T' is connected.

Kruskal Algorithm: Proof of optimality

- **T**: MST found by Kruskal Algorithm
- **M**: optimal MST

Proof by contradiction. Suppose T ≠ M.

$T = e_1\ e_{2\,.}\ \dots\ e_j\ \dots\ e_n$

$M = e_1\ e_{2.}\dots\ m_j\ \dots\ m_n$

- T and M are the same up to j-th edge. Suppose e_j=(u, v) is in T but not in M
- C: the connected component containing u when (u,v) was added to T
- When (u,v) was added, it was the least-cost edge crossing the cut (C, V-C)
  - (u, v) crosses the cut, since u and v were not connected when Kruskal's algorithm selected (u, v)
  - Kruskal algorithm selects the least-cost edge crossing the cut
- **M** is a MST → There must be a path from u to v in **M**. This path begins in C and ends in V-C. → There must be an edge along that path where x in C and y in V-C. Since (u,v) is the least-cost edge crossing (C, V-C) → **w(u,v) < w(x,y)**
- M' = M-{(x,y)} U {(u,v)}. M' is a spanning tree because it connects all vertices. Since (x,y) is on the cycle formed by adding (u, v)
- w(M') = w(M) - w(x,y) + w(u,y) < w(M) → M' is a MST → contradiction, M was the optimal solution
- We used exchange argument
  - exchanging some part of the optimal solution with some part of the greedy solution improved the optimal solution → contradiction
- Note: here we are assuming the edge weights are **unique**, otherwise we do not reach a contradiction

Kruskal Algorithm: pseudocode

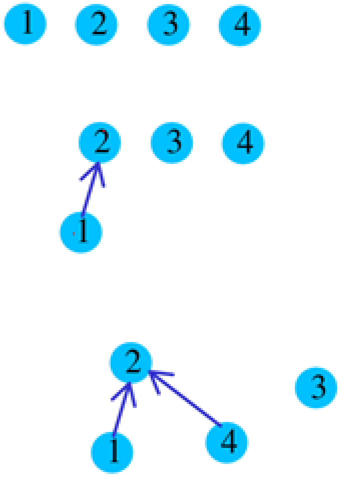
```
Kruskal(G)
    Sort the edges by non-decreasing weight $e_1$ … $e_m$, $w(e_i) \leq w(e_{i+1})$
    T = ∅
    for each edge (u, v)
        if u and v are not connected by T
            T = T ∪ {(u,v)}
    return T
```

Union-find data structure

- Purpose: Divides a set of data, $S = \{e_1, e_2, \ldots, e_n\}$, into **disjoint subsets**.
    - a collection of disjoint sets $\{S_1, S_2, \ldots, S_k\}$
        - Each element of data belong to exactly one set
    - Initially n disjoint subsets $S_i = \{e_i\}$
    - Each set is identified by a representative (some member of the set)
        - Specifies which set an element belongs to
- Operations of **union-find** data structure
    - **Make-set**(x): Create a set containing one element, x
    - **union**(x, y): unites the sets containing x and y into one set
    - **find**(x): returns a pointer to the representative of the set containing x

Union-find data structure

- Each set is represented as a tree of pointers
    - Initially one single-node tree per subset
    - Each node has a parent pointer



find(1)->1, find(2)->2

Union(1,2): same height, parent[1] = 2
find(4)->4, find(1)->2

Union(4,2):2's height is greater: parent[4] = 2

- $Find(i)$ returns the **root** of the tree containing **element *i***
    - To check whether i and j are in the same component, just compare the result of find
- $Union(i,j)$ merges two connected components
    - makes the root with greater heights to be the parent of the other

Kruskal Algorithm using union-find data structure

```
Kruskal(G)
    Sort the edges by non-decreasing weight $e_1 … e_m$, $w(e_i) \leq w(e_{i+1})$
    T = Ø
    S = union-find data structure
    for each v in V
        S.make-set(v)
    for each edge (u, v)
        if S.find(u) != S.find(v) # u and v are in different sets
            T = T U {(u,v)}
            S.union(u, v)     # merge the sets containing u and v
    return T
```

Runtime: O(E log V)

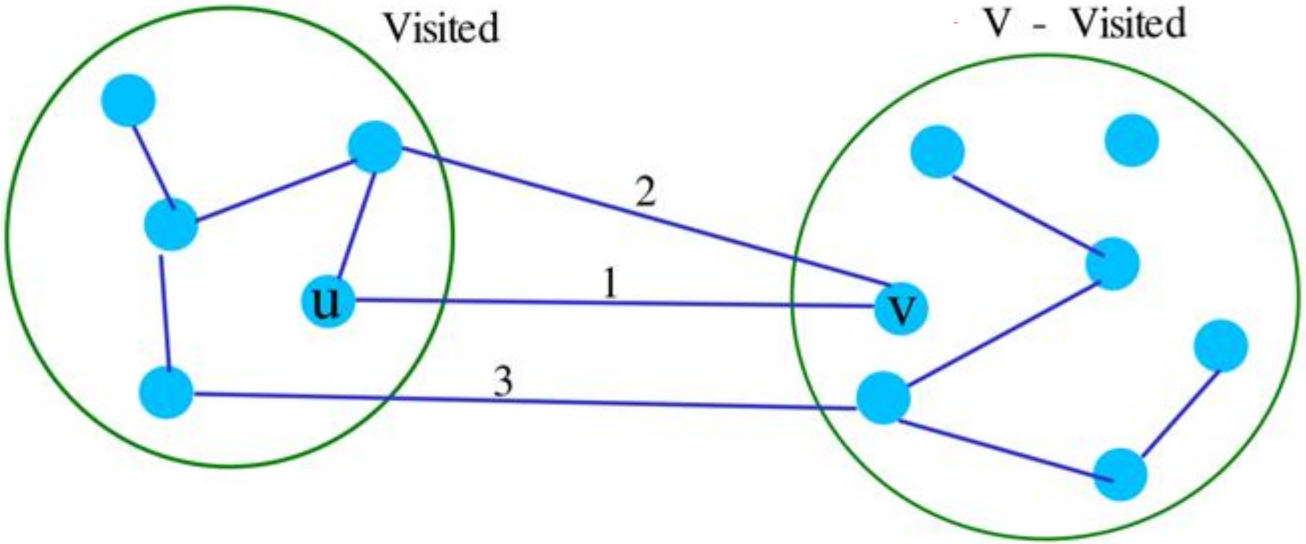
Runtime of find operation

- Each tree of height h contains at least 2^h nodes
- There are only n nodes in the graph
- Therefore the height is at most log n
- The longest path in the union-find first is log n
- So all find operations run in Θ (log n) time

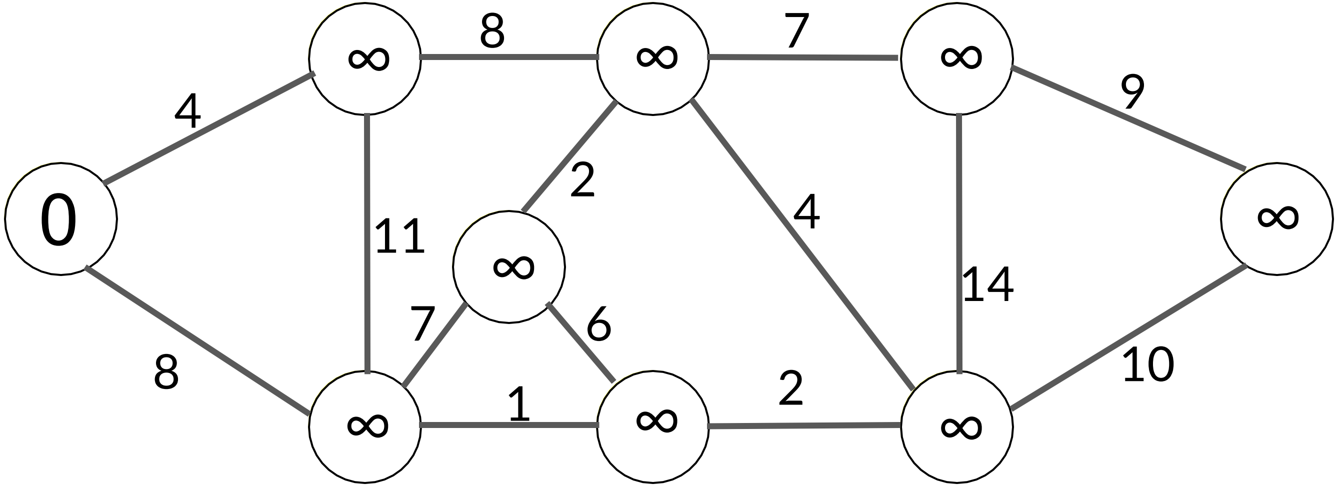
Prim’s Algorithm

Prim’s Algorithm

Idea: Grow one connected component in a greedy fashion (i.e., by adding a vertex **v ∈ V − Visited** that is one end of a minimum weight edge leaving Visited).

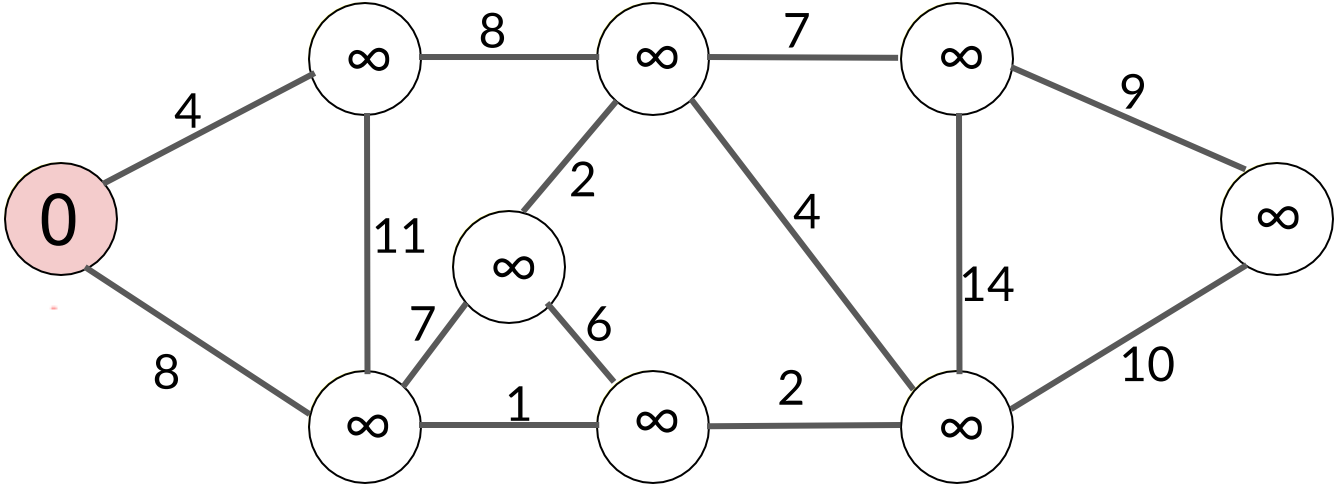

Visited
V - Visited
2
1
3
u
v

Prim's algorithm



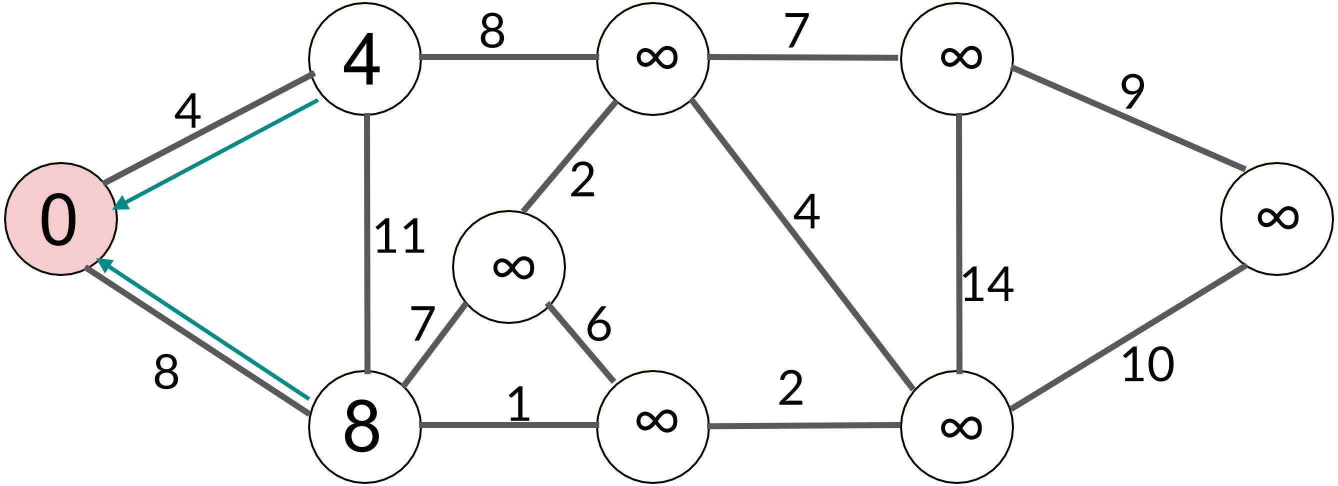
0
∞
∞
∞
∞
∞
∞
∞
∞
4
8
8
7
9
11
2
4
14
7
6
1
2
10

Prim's algorithm



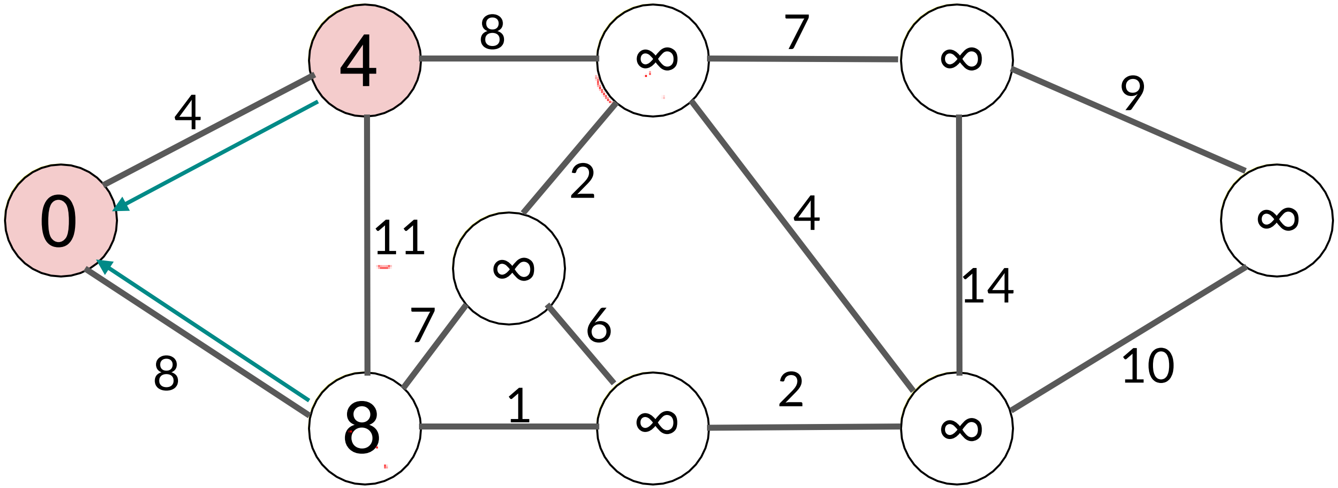
8
7
∞
∞
∞
4
9
2
0
4
∞
11
∞
14
7
6
8
10
1
2
∞
∞
∞

Prim's algorithm

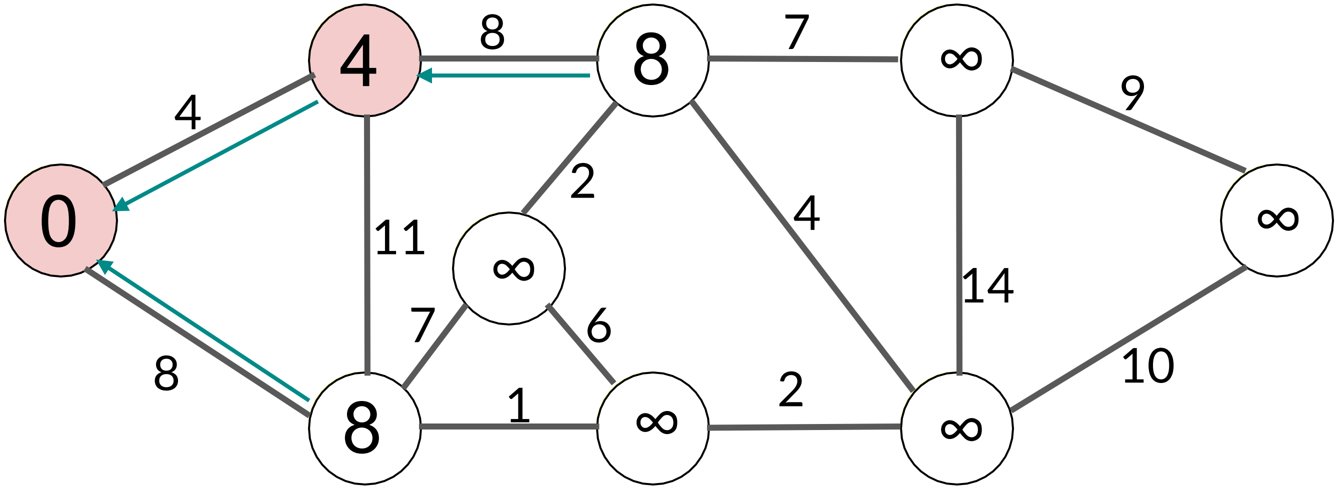


0
4
8
∞
∞
∞
∞
∞
∞
4
8
8
7
9
11
2
4
14
7
6
1
2
10

Prim’s algorithm

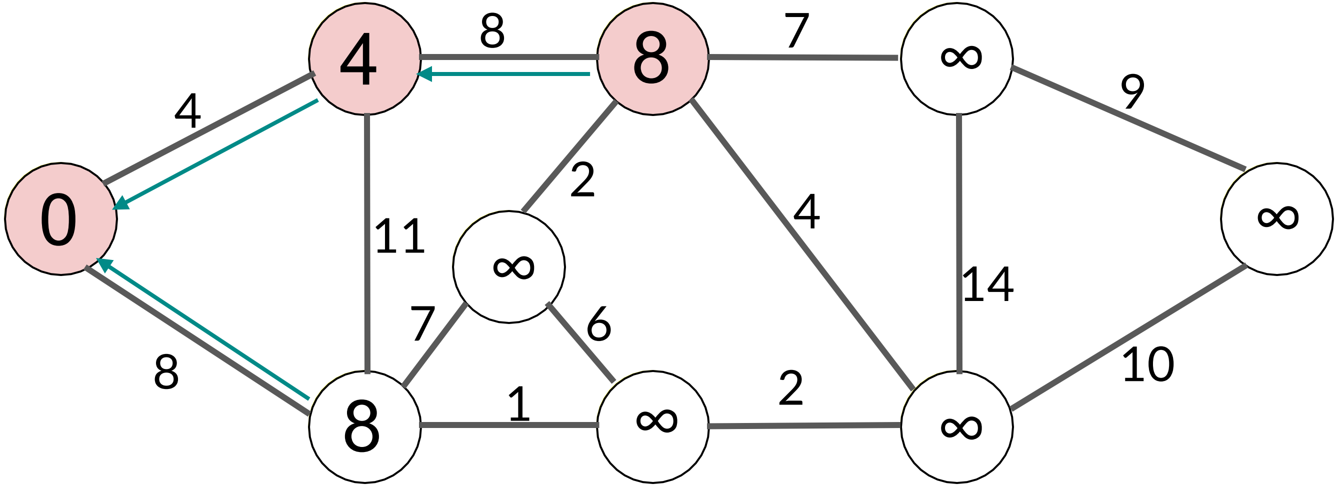

4
8
7
9
4
0
11
2
4
∞
∞
∞
14
∞
7
6
8
10
8
1
2
∞
∞
∞

Prim’s algorithm



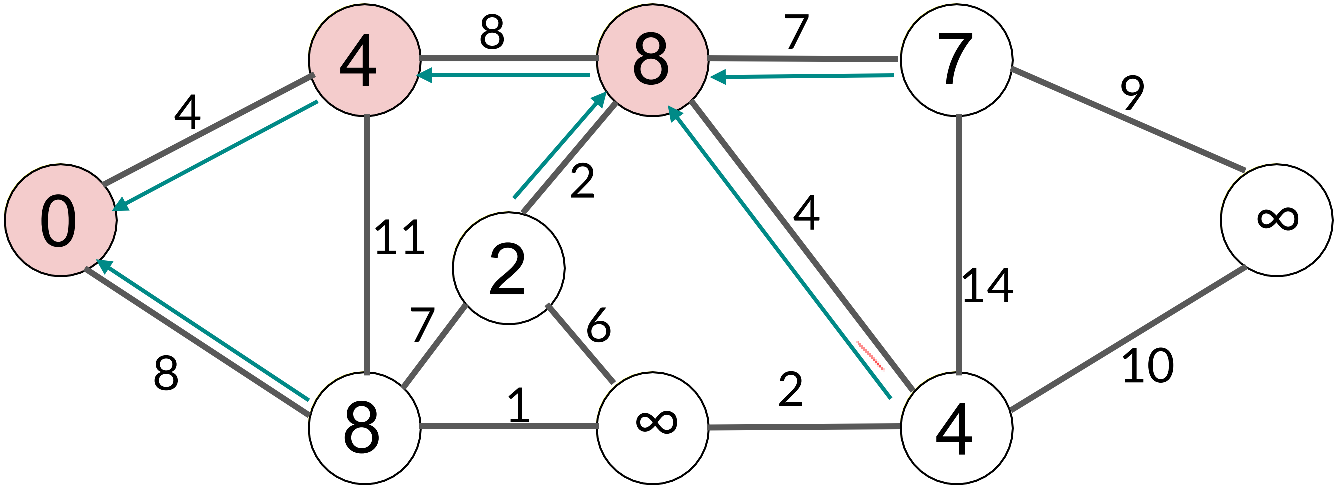
0
4
8
8
∞
∞
∞
∞
∞
4
8
8
7
9
11
2
4
14
7
6
1
2
10

Prim's algorithm

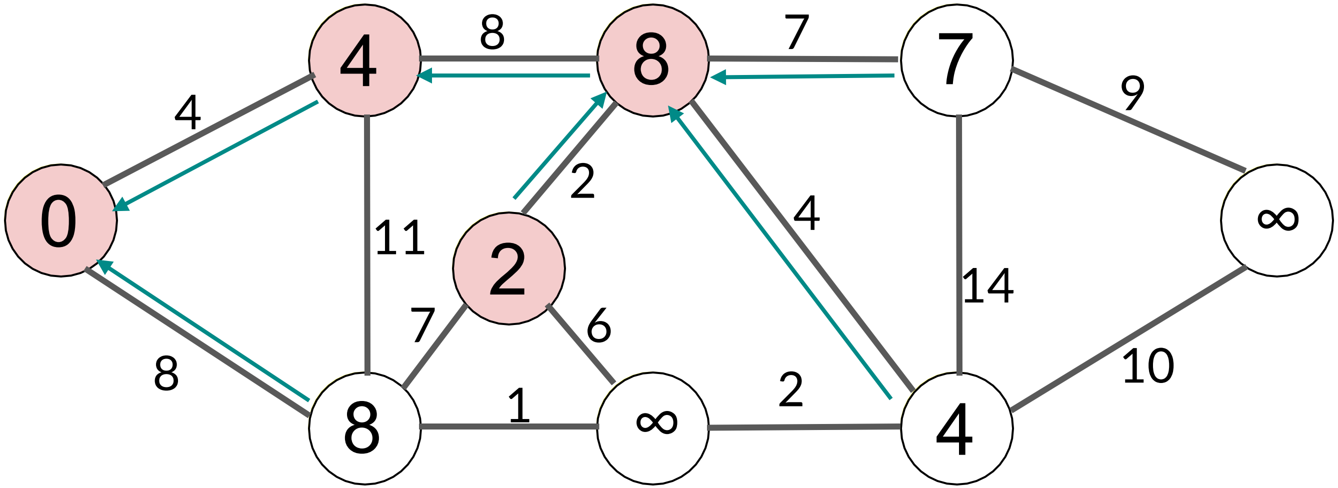


4
8
8
7
∞
9
4
0
11
2
∞
4
∞
14
7
6
8
8
1
∞
2
∞
10

Prim’s algorithm

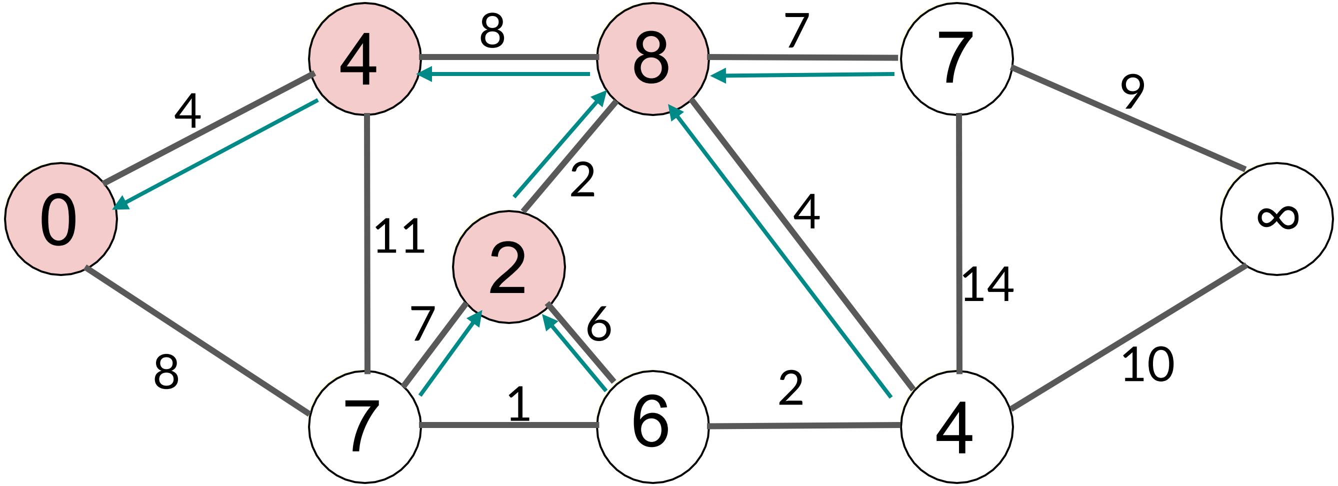

4
8
7
0
4
8
7
9
∞
11
2
2
4
14
7
6
8
1
2
10
8
∞
4

Prim’s algorithm



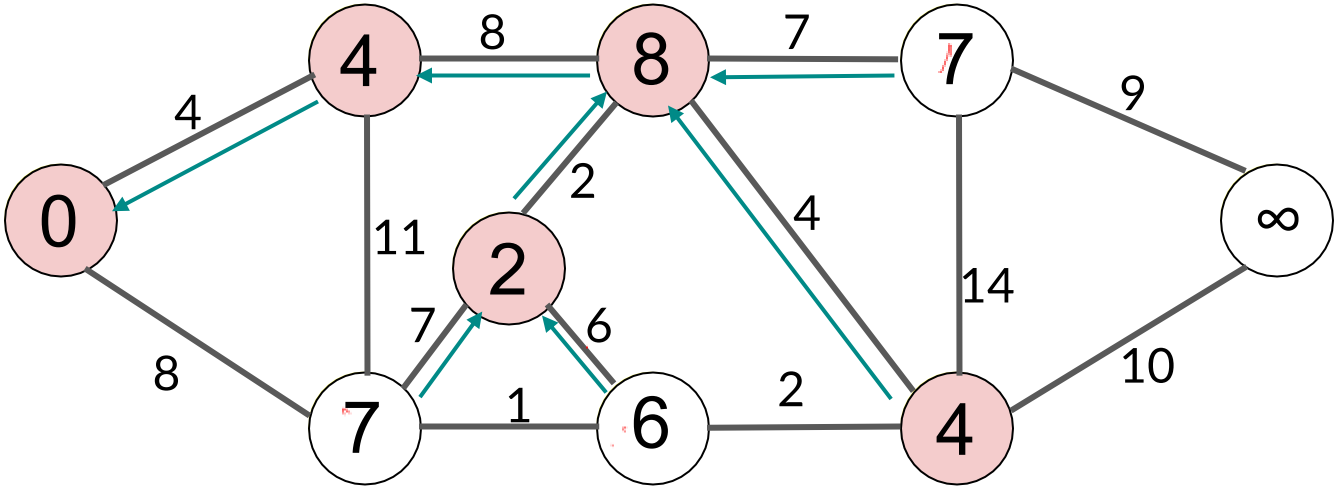
4
8
7
7
9
0
4
11
2
2
4
8
∞
14
7
6
8
1
∞
2
4
10
8

Prim’s algorithm

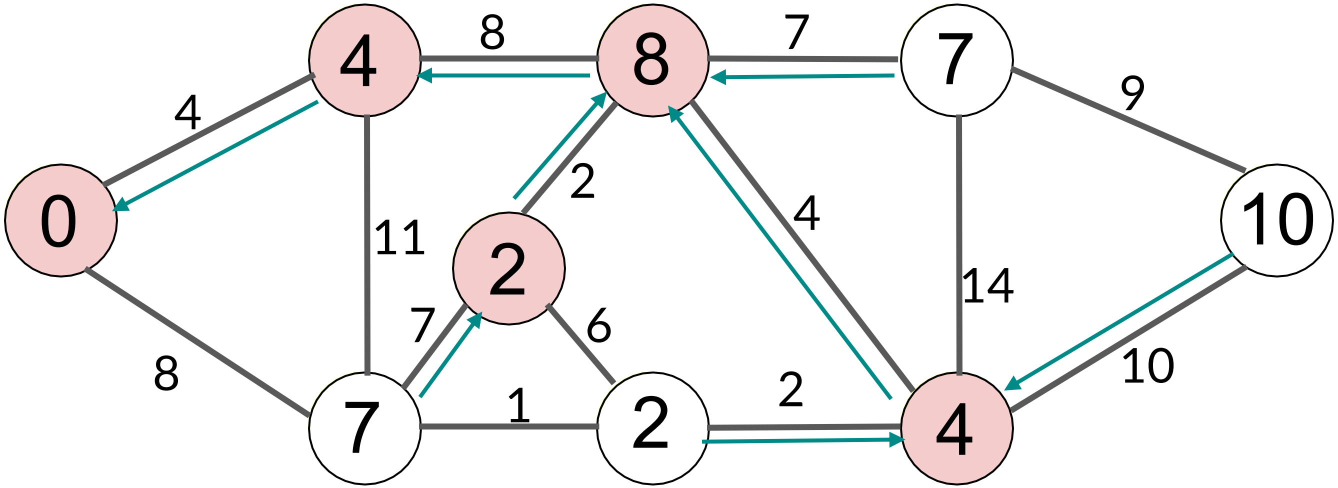


4
8
7
0
11
2
2
7
9
∞
4
14
7
6
8
7
1
6
2
4
10

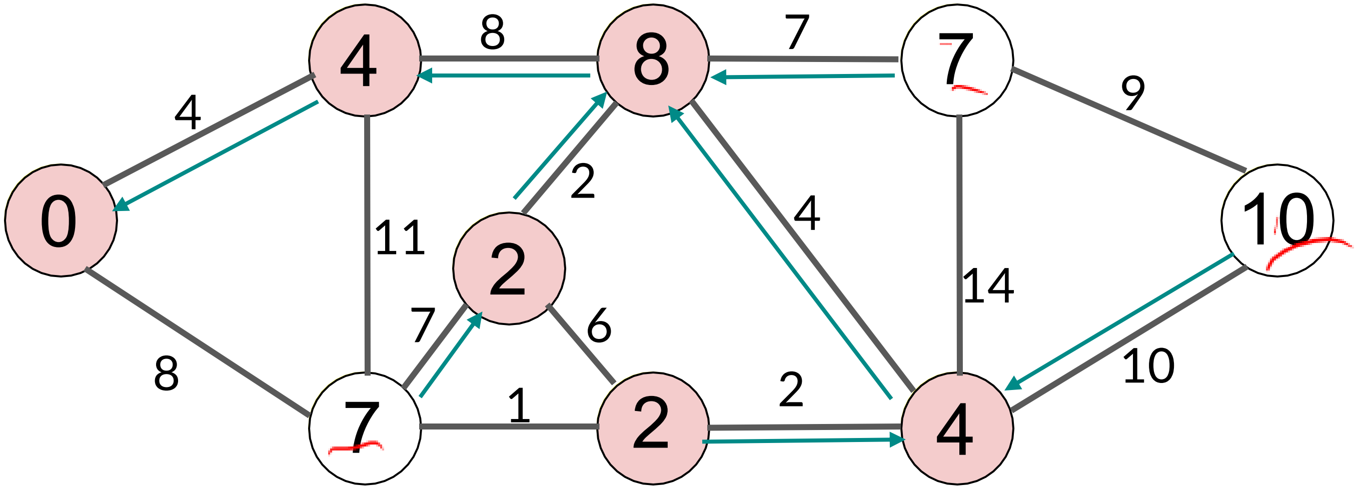
Prim’s algorithm


4
8
7
0
4
8
7
9
11
2
4
∞
2
7
6
14
8
1
2
10
7
6
4

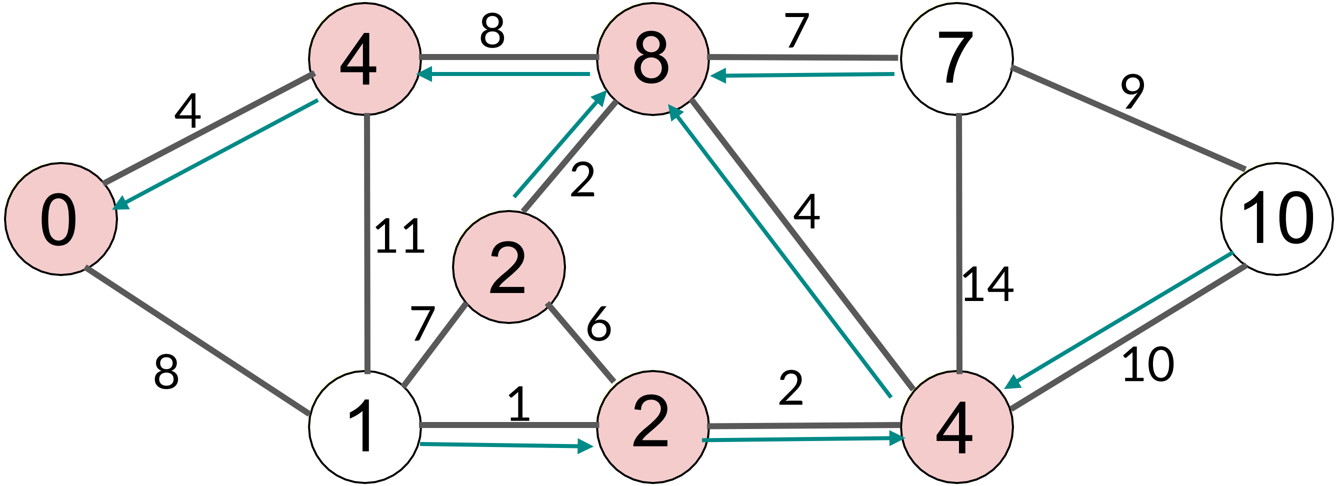
Prim's algorithm


4
8
8
7
7
9
4
0
2
11
2
10
4
14
7
6
8
7
1
2
2
10
4

Prim’s algorithm

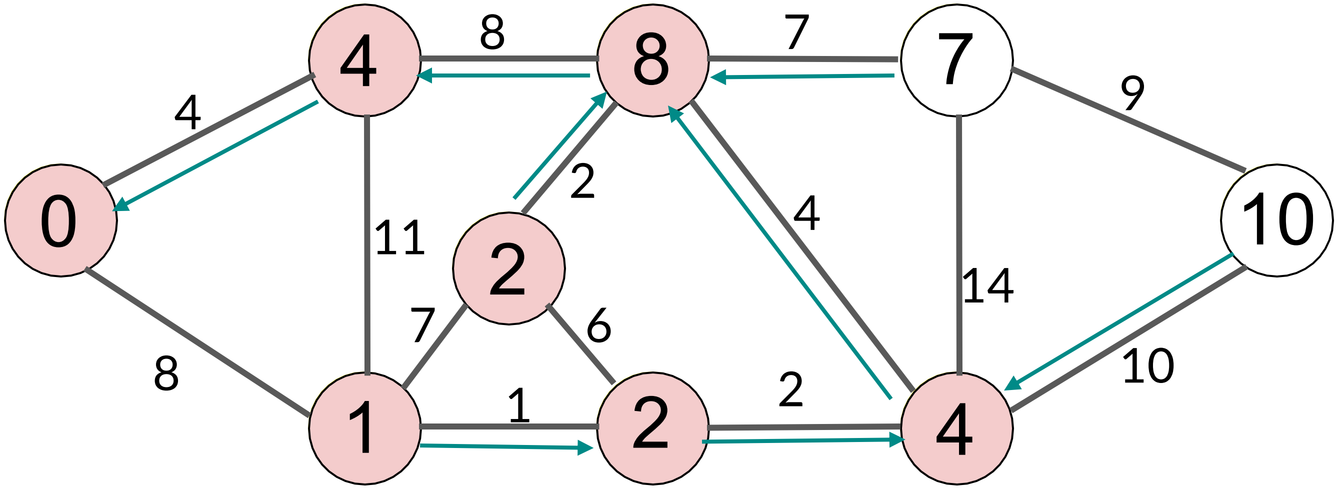

4
8
8
7
7
9
4
0
11
2
2
4
10
7
6
14
8
7
1
2
2
4
10

Prim’s algorithm



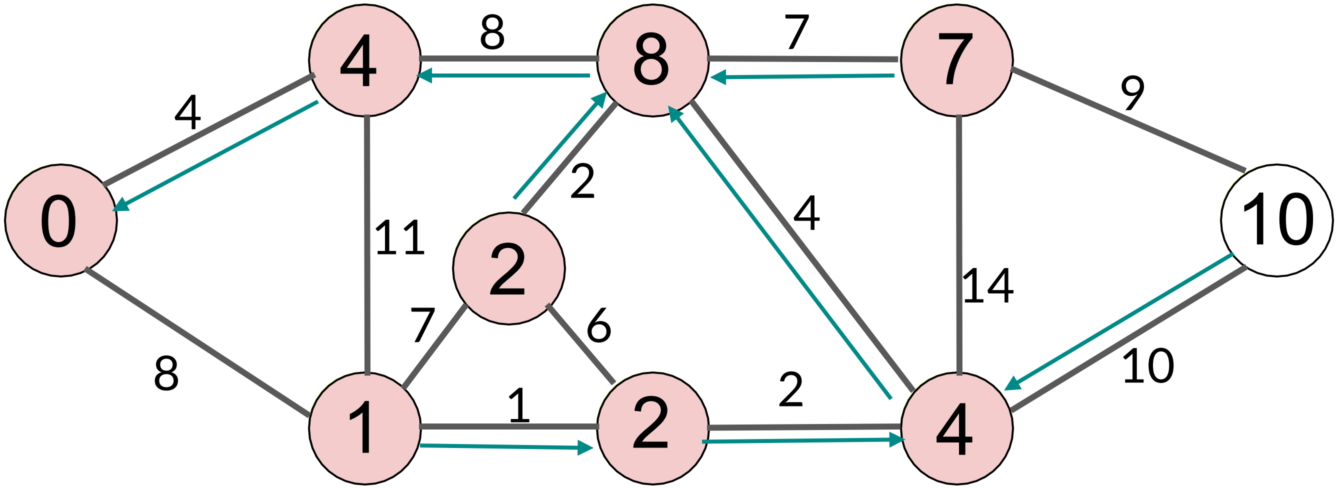
4
8
8
7
7
4
0
9
11
2
2
4
10
14
7
6
8
1
2
2
4
10
1

Prim’s algorithm

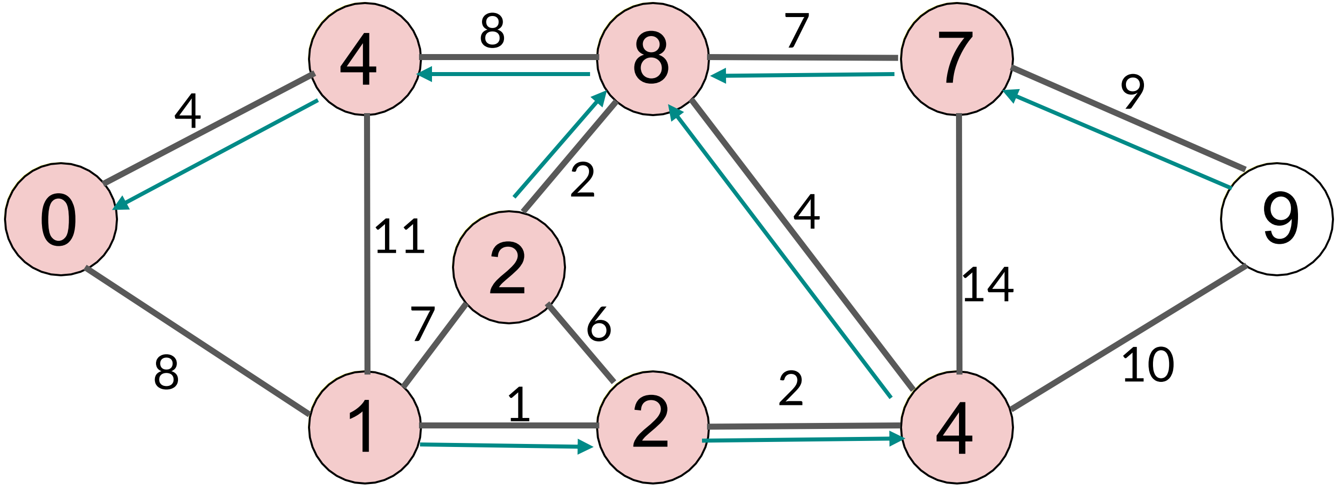


4
8
7
7
9
4
0
11
2
2
4
10
14
7
6
8
1
2
2
10
1
2
4

Prim’s algorithm

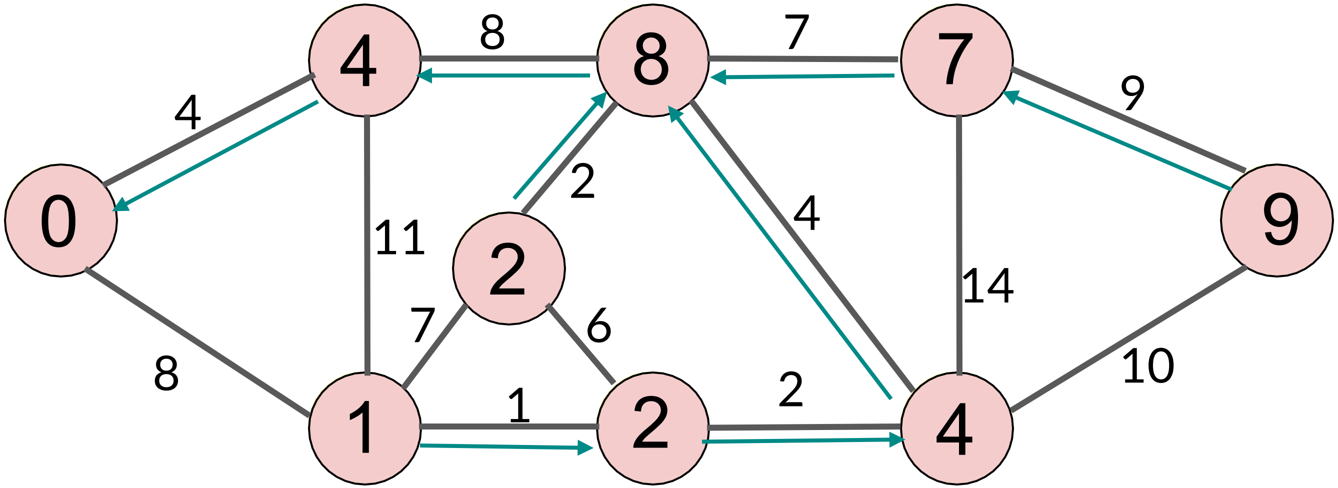

4
8
8
7
7
9
0
4
2
11
10
2
4
14
7
6
8
10
1
1
2
2
4

Prim's algorithm



4
8
7
7
8
4
9
0
11
2
2
4
9
14
7
6
8
1
1
2
2
4
10

Prim’s algorithm


4
8
8
7
7
4
9
0
4
2
11
2
4
14
9
7
6
8
1
10
1
2
2
4

Prim’s Algorithm: Correctness: Unique edge weights

- **T**: MST found by Prim’s Algorithm
- **M**: optimal MST

Proof by contradiction. Assume T ≠ M →T-M ≠ Ø → Let (u, v) be any edge in T - M.

- When (u,v) was added, it was the least-cost edge crossing the cut (Visited, V-Visited)
  - (u, v) crosses the cut, since u and v were not connected when Priml's algorithm selected (u, v)
  - Prim’s algorithm select the least-cost edge crossing the cut
- **M** is a MST → There must be a path from u to v in **M**. This path begins in visited and ends in V-Visited. → There must be an edge along that path where x ∈ Visited and y ∈ V-Visited. Since (u,v) is the least-code edge crossing (Visited, V-Visited) → **w(u,v) < w(x,y)**
- M' = M-{(x,y)} U {(u,v)}. M' is a spanning tree because it connects all vertices. Since (x,y) is on the cycle formed by adding (u, v)
- w(M') = w(M) - w(x,y) + w(u,v) < w(M) → M' is a MST → contradiction M was the optimal solution

Prim’s Algorithm

```
Prim-simple(G, s)

    T = ∅
    Visited = {s}
    while Visited ≠ V
         find vertex v ∉ Visited such that
         there exists a u ∈ visited and
         (u,v) is a minimum weight edge leaving Visited

         T = T U {(u, v)}
         Visited = Visited U {v}
    return T
```

Greedy choice: at each step it adds to the tree an edge that contributes the minimum amount possible to the tree’s weight
Choose vertex v ∈ V − visited connected to a minimum weight edge e = (u, v) between Visited and V − Visited

Prim’s Algorithm: better implementation

- **Idea**: Maintain V – Visited as a priority queue Q.
- For v ∊ V- Visited, we define:

$$\mathrm{weight(v)} = \begin{cases} \infty \\ \min \mathrm{w(e)} \mid \mathrm{e} = \mathrm{(u, v)} \in \mathrm{E} \text{ and } u \in T \end{cases}$$

- The weight of each vertex in V-Visited is the weight of the least-weight edge connecting it to a vertex in Visited.
- Priority Queue implemented using heap data structure
  - V - Visited is maintained as an array in heap order, and the key of each vertex is its weight defined above
  - ExtractMin(): remove and return vertex with minimum weight
  - Insert(v, weight(v)): insert vertex v with weight(v)
  - DeleteMin(v): delete the vertex with minimum weight
  - decrease-key(v, oldWeight, newWeight)
    - deletes vertex V with oldWeight and inserts vertex V with newWeight
  - The runtime of all operations are O(log k) where k is the size of heap

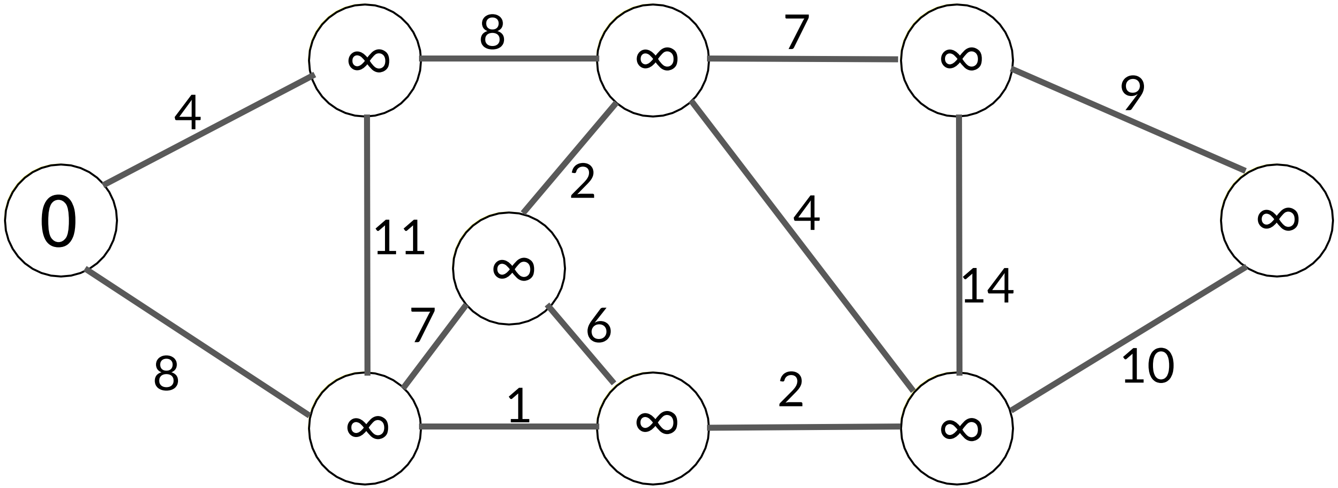
Prim’s Algorithm: better implementation

Idea: Maintain V – Visited as a priority queue Q. The key of each vertex in Q is the weight of the least-weight edge connecting it to a vertex in T.

```
Prim(G, s)
    T = Ø
    Q = V
    Key[s] = 0
    Key[u] = ∞ for all u ∈ V
    while Q ≠ Ø
        u = EXTRACT-MIN(Q)
        for each v ∈ Adj[u]
            if v ∈ Q and w(u,v) < key[v]
                Key[v] = w(u, v)
                pred[v] = u # T = T ∪ {(u, v)}
    return T
```

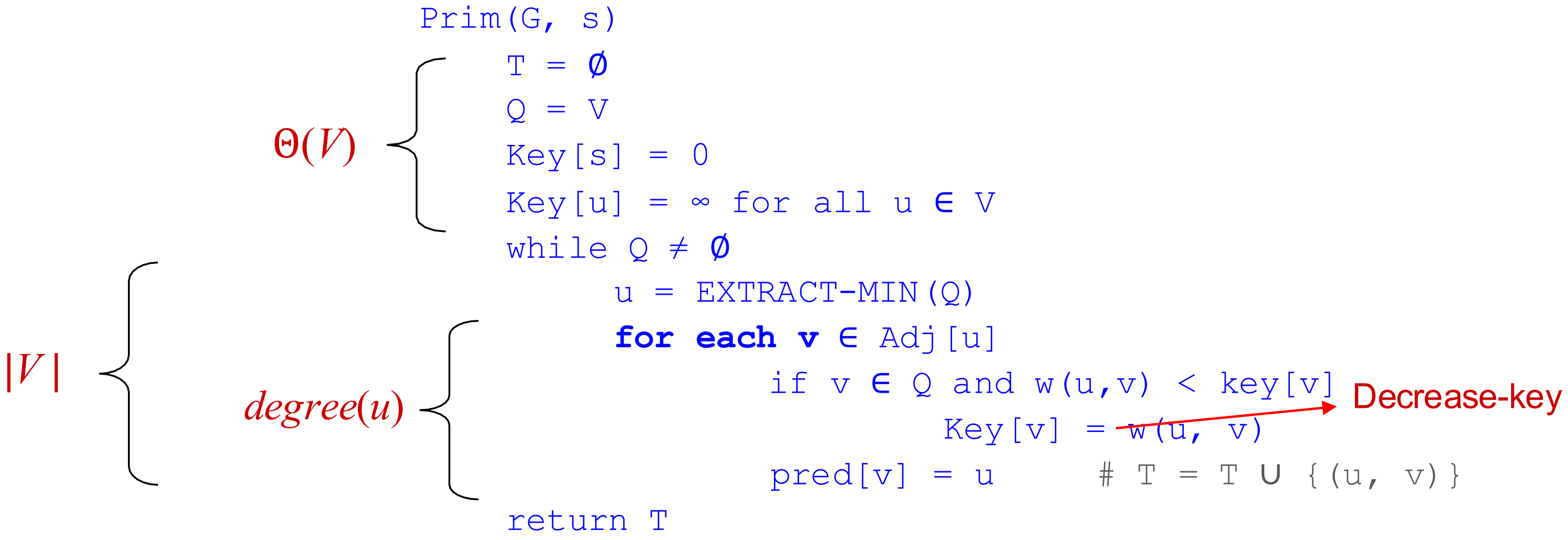
At the end, {(v, pred[v])} forms the MST.

Prim’s algorithm


0
∞
∞
∞
∞
∞
∞
∞
∞
4
8
11
8
7
2
7
6
1
2
4
9
14
10

Prim’s Algorithm: Runtime

Idea: Maintain V – T as a priority queue Q. The key of each vertex in Q is the weight of the least-weight edge connecting it to a vertex in T.



```
Prim(G, s)
    T = ∅
    Q = V
    Key[s] = 0
    Key[u] = ∞ for all u ∈ V
    while Q ≠ ∅
        u = EXTRACT-MIN(Q)
        for each v ∈ Adj[u]
            if v ∈ Q and w(u,v) < key[v]
                Key[v] = w(u, v)
            pred[v] = u     # T = T ∪ {(u, v)}
    return T
```

$$\text{Runtime} = \Theta(V)\cdot(T_{\text{EXTRACT-MIN}}) + \Theta(E)\cdot(T_{\text{DECREASE-KEY}})$$

Prim's Algorithm: Runtime

Runtime = $\Theta(V) \cdot (T_{\text{EXTRACT-MIN}}) + \Theta(E) \cdot (T_{\text{DECREASE-KEY}})$

Q	$T_{\text{EXTRACT-MIN}}$	$T_{\text{DECREASE-KEY}}$	Total Time
array	$O(V)$	$O(1)$	$O(V^2)$
Binary heap	$O(\lg V)$	$O(\lg \mathrm{V})$	$O(\mathrm{E} \lg \mathrm{V})$